

Sharding

29 August 2026 01:43 PM

You can add indexes, tune queries, add read replicas, and buy a larger server. Those are usually the right first moves. But if one primary database can no longer handle the writes, storage, or hot data in memory, you may need to split the data across multiple machines.

That is **sharding**.

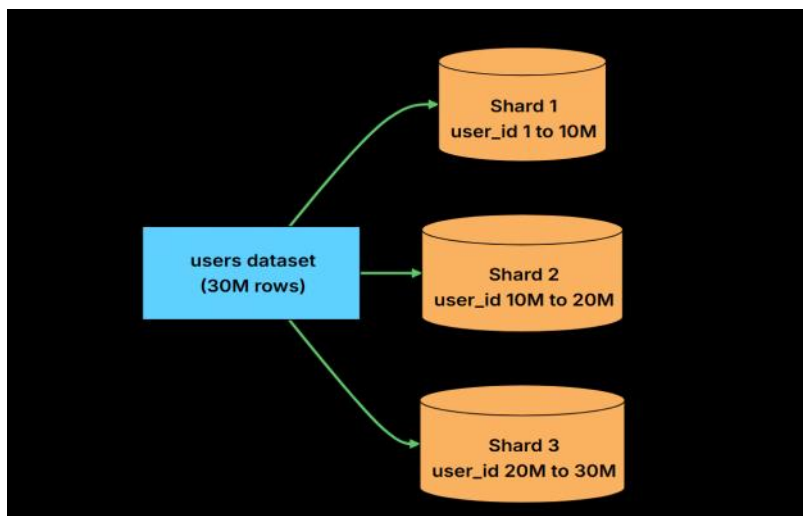
Sharding means splitting rows across multiple database nodes. Each node stores only part of the data. Together, the shards still represent one complete dataset, but no single database server owns all the rows.

Sharding can scale writes and storage, but it is one of the most expensive database scaling decisions you can make. It spreads data and load across machines, but it also makes routing, transactions, joins, migrations, and rebalancing harder. Use it when the bottleneck is real and simpler techniques are no longer enough.

1. What is Database Sharding?

Database sharding splits a dataset into smaller pieces called **shards**.

Each shard stores part of the data and usually runs on a separate database server or cluster.



For example, users might be split by `user_id`:

Shard	Users
Shard 1	<code>user_id</code> 1 to 10M
Shard 2	<code>user_id</code> 10M to 20M
Shard 3	<code>user_id</code> 20M to 30M

When the application needs user 18M, it routes the query to Shard 2.

The column used to choose the shard is the **shard key**. Picking the shard key is the most important design decision in sharding, because it decides where rows live and which queries stay simple.

2. Why Shard?

Sharding helps when one primary database cannot handle the workload.

Common reasons:

- **Storage:** the dataset no longer fits comfortably on one machine.
- **Write volume:** one primary cannot process all writes.
- **Hot data size:** the frequently used data no longer fits in memory on one node.
- **Customer isolation:** large customers need their own capacity.
- **Regional placement:** data needs to live near users or inside legal boundaries.

3. How Sharding Works

A sharded system needs three things:

1. A shard key.
2. A mapping from shard key to shard.
3. A router that sends each request to the right shard.

The router may live in application code, a data-access library, a proxy, or the database system itself.

For a request like:

the router can use `user_id = 123` to find the correct shard.

If the query does not include the shard key, routing becomes harder. The system may need to ask every shard and merge the results. That is called a **scatter-gather query**. It is one of the major costs of sharding.

4. Choosing a Shard Key

The shard key decides where data lives.

A good shard key spreads both data and traffic evenly across shards so no single shard becomes overloaded. It also matches the queries the application runs most often. When possible, it keeps related data together so common operations stay on one shard. It should leave room to grow as the dataset changes over time.

This is harder than it sounds, and it is where many sharding projects succeed or fail.

Good Shard Key: `user_id`

For many consumer applications, `user_id` is a reasonable shard key.

Most user-specific queries include the user ID:

leanly to one shard.

Risky Shard Key: `created_at`

Sharding by timestamp can create hot shards.

If all new writes go to the shard for "today" or "this month," one shard receives most writes while older shards sit idle.

Time-based sharding can work for append-heavy logs or archive systems, but it needs careful design around hot ranges and old data cleanup.

Risky Shard Key: Fields With Few Values

Fields like `country`, `status`, or `plan_type` usually have too few possible values.

If you shard by `country`, one large country may dominate the system. If you shard by `status`, almost all active users may live on one shard.

Shard keys with only a few values often produce uneven data and uneven traffic.

5. Sharding Strategies

Sharding = splitting data across multiple databases/servers (shards).

The main question is: **How do we decide which shard gets a particular piece of data?**

There are 4 common strategies:

1. Range-Based Sharding

Data is divided according to **ranges of the shard key**.

Example:

Shard **User ID Range**

Shard 1 1 – 10M

Shard 2 10M – 20M

Shard 3 20M – 30M

So:

`user_id = 15M`

↓

10M–20M

↓

Shard 2

Advantages

- Simple to understand.
- Good for **range queries**.

Example:

`WHERE user_id BETWEEN 10M AND 20M`

Disadvantage

Hotspot problem.

If new users always get higher IDs, the last shard may receive most writes.

Shard 1 → 100 requests

Shard 2 → 100 requests

Shard 3 → 1000 requests 📍

2. Hash-Based Sharding

Apply a **hash function** to the shard key.

`hash(user_id) % number_of_shards`

Example:

`user_id`

↓

`hash()`

↓

`% 4`

↓

Shard 0/1/2/3

Advantage

Usually distributes data **more evenly**.

Disadvantage

Rebalancing is difficult.

If you change:

4 shards → 5 shards

many keys may map to different shards.

Solution

Production systems often use:

- **Virtual shards/buckets**
- **Consistent hashing**

These reduce the amount of data that needs to move.

3. Directory-Based Sharding

Maintain a **directory/lookup table** that tells us where each key belongs.

Example:

Tenant	Shard
acme	Shard 7
globex	Shard 2
initech	Shard 11

When request comes:

tenant = acme

↓

Directory

↓

Shard 7

Advantage

Very **flexible**.

You can move one tenant:

acme → Shard 7

to:

acme → Shard 15

by changing the directory.

Disadvantage

The **directory becomes critical**.

If directory is down/wrong → routing can fail.

So it must be:

- Highly available
- Carefully cached
- Safely updated during migration

4. Geo-Based Sharding

Data is divided based on **geographical region**.

Example:

US users → US shards

EU users → EU shards

India users → India shards

Advantages

- Lower latency for users.
- Helps satisfy **data-location/regulatory requirements**.

Disadvantages

- Global queries become difficult.
- Handling users who move between regions becomes complicated.

6. Cross-Shard Queries

Most important rule:

Try to include the shard key in your query.

Suppose data is sharded by:

user_id

This is good:

SELECT * FROM orders

WHERE user_id = 123;

The router knows:

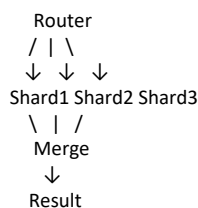
user_id = 123

↓

Shard 2
Only **one shard** is contacted.

Problem: Query without shard key

SELECT * FROM orders
WHERE status = 'pending';
If status isn't the shard key, the system may need to ask:



This is called **Scatter-Gather**.

Scatter-Gather

Scatter: Send query to many/all shards.

Gather: Collect and merge results.

Problems

- Slower
- More expensive
- More complex
- Gets worse as number of shards increases

Better approach

Design APIs/queries around the **shard key**.

For truly global queries, use:

- Search index
- Read models
- Analytics system

instead of querying every shard every time.

7. Joins and Transactions Across Shards

Joins

Same shard → easy

Example:

orders
order_items
Both are sharded by:

user_id
So a user's orders and order items are on the same shard.

user_id = 101
↓
Shard 2
↓
orders + order_items
Local join is easy.

Different shard keys → difficult

Suppose:

orders → shard by user_id
products → shard by product_id
Now an order may be on one shard and its product on another.
This requires **cross-shard coordination** or copying some data.

Important design principle

Keep related data on the same shard whenever possible.

Transactions

Transaction inside one shard

Easy:

Transaction
↓
Shard 1
Normal database transaction can be used.

Transaction across multiple shards

Transaction

↓

Shard 1

Shard 2

Shard 3

Much harder.

Requires:

- Distributed transactions, OR
- Application-level rollback/repair

This adds:

- Latency
- Complexity
- Failure cases

Key point

Keep important transactions within one shard whenever possible.

8. Rebalancing / Resharding

Rebalancing = moving data between shards to make the load more balanced.

Example:

Before:

Shard 1 → 100 GB

Shard 2 → 100 GB

Shard 3 → 500 GB 🔴

We want:

After:

Shard 1 → 250 GB

Shard 2 → 225 GB

Shard 3 → 225 GB

Why is it difficult?

While moving data:

Old Shard —————→ New Shard

↓

Copy data

↓

Keep data synchronized

↓

Move traffic

↓

Verify

↓

Delete old data

The system must correctly handle **reads and writes during migration**.

Techniques

- Virtual shards/buckets
- Consistent hashing
- Directory mapping
- Splitting hot ranges

Important point

Build migration/rebalancing tools **before** you urgently need them.

9. Hot Shards

A **hot shard** is a shard receiving much more traffic/data than others.

Example:

Shard 1 → 100 requests

Shard 2 → 120 requests

Shard 3 → 5000 requests 🔴

The entire system can become limited by this one shard.

Common causes

- Bad shard key
- Huge tenant
- Celebrity/high-traffic user
- Popular product
- Time-based key
- Hot range
- Uneven regional traffic

Solutions

1. Move large tenants

Normal shard → Dedicated shard

2. Split very hot entities

Use another/split key to distribute their traffic.

3. Add caching

Useful for hot reads.

4. Buffer writes

Useful for bursty traffic.

5. Split hot ranges

Useful in range-based sharding.

6. Reconsider the data model

If one entity always becomes hot, the problem may be the data design itself.

10. When NOT to Shard

This is **very important for interviews**.

Don't shard just because sharding sounds scalable.

First try simpler solutions.

Step 1: Fix indexes

Bad query/index problems can cause major performance issues.

Step 2: Optimize queries

Check slow queries and query plans.

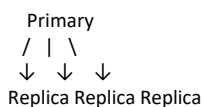
Step 3: Add caching

If reads are the problem:

Application → Cache → Database

Step 4: Read replicas

For read-heavy workloads:



Step 5: Archive old data

Move cold/old data away.

Step 6: Scale vertically

Use a bigger database machine if there is still room to grow.

Step 7: Precomputed read models

Instead of running expensive queries repeatedly, prepare the data in advance.